

## 5. Stringek kezelése C#-ban

---

### Mi az a string?

A **string** egy beépített referencia típus C#-ban, amely karakterláncokat (szövegeket) tárol. A string **immutable** (megváltozhatatlan), ami azt jelenti, hogy egy létrehozott string értéke nem módosítható - minden módosítás új string objektumot hoz létre.

```
string uzenet = "Hello, World!";
```

---

### String létrehozása

#### Alapvető módok

```
// 1. String literál
string nev = "Kovács János";

// 2. Üres string
string ures1 = "";
string ures2 = string.Empty; // Ajánlott módszer

// 3. Null érték
string nullString = null;

// 4. String konstruktor karaktertömbből
char[] betuk = { 'H', 'e', 'l', 'l', 'o' };
string szo = new string(betuk); // "Hello"

// 5. Karakter ismétlése
string csillagok = new string('*', 10); // "*****"

// 6. String.Concat
string ossz1 = string.Concat("Hello", " ", "World"); // "Hello World"

// 7. String.Join
string[] szavak = { "C#", "Java", "Python" };
string nyelvek = string.Join(", ", szavak); // "C#, Java, Python"
```

---

### String tulajdonságok

#### Length - string hossza

```
string nev = "Anna";
int hossz = nev.Length; // 4

string ures = "";
Console.WriteLine(ures.Length); // 0

// Gyakori használat
string jelszo = "abc123";
if (jelszo.Length >= 8)
{
    Console.WriteLine("Jelszó elég hosszú.");
}
else
{
    Console.WriteLine($"A jelszó túl rövid! ({jelszo.Length}/8 karakter)");
}

// Null ellenőrzés fontos!
string nullStr = null;
// Console.WriteLine(nullStr.Length); // NullReferenceException!

// Biztonságos ellenőrzés
if (nullStr != null && nullStr.Length > 0)
{
    Console.WriteLine("Van tartalom");
}

// Vagy
if (!string.IsNullOrEmpty(nullStr))
{
    Console.WriteLine("Van tartalom");
}
```

---

## String indexelés

A stringek karaktereihez index segítségével férhetünk hozzá (0-tól kezdődik).

```
string szo = "Hello";

// Karakter lekérése index alapján
char elso = szo[0]; // 'H'
char masodik = szo[1]; // 'e'
char utolso = szo[4]; // 'o'

// Utolsó karakter
char utolso2 = szo[szo.Length - 1]; // 'o'

Console.WriteLine($"Első betű: {elso}");
```

```
// Bejárás indexszel
string nev = "Anna";
for (int i = 0; i < nev.Length; i++)
{
    Console.WriteLine($"Index {i}: {nev[i]}");
}
/* Kimenet:
Index 0: A
Index 1: n
Index 2: n
Index 3: a
*/

// FIGYELEM: Nem módosítható!
string szoveg = "Hello";
// szoveg[0] = 'M'; // HIBA! String immutable

// Ha módosítani akarunk, új stringet kell létrehozni
string ujSzoveg = 'M' + szoveg.Substring(1); // "Mello"

// Index túllépés
string s = "ABC";
// char c = s[10]; // IndexOutOfRangeException!

// Biztonságos hozzáférés
if (i < s.Length)
{
    char c = s[i];
}
```

---

## String összehasonlítás

### Egyenlőség vizsgálat

```
string s1 = "hello";
string s2 = "hello";
string s3 = "HELLO";

// == operátor (kis/nagybetű érzékeny)
bool egyenlo1 = (s1 == s2); // true
bool egyenlo2 = (s1 == s3); // false

// Equals metódus
bool egyenlo3 = s1.Equals(s2); // true
bool egyenlo4 = s1.Equals(s3); // false

// Kis/nagybetű független összehasonlítás
bool egyenlo5 = s1.Equals(s3, StringComparison.OrdinalIgnoreCase); // true

// ToLower/ToUpper használata
```

```
bool egyenlo6 = s1.ToLower() == s3.ToLower(); // true

// String.Compare
int eredmeny = string.Compare(s1, s2);
// eredmeny < 0: s1 < s2
// eredmeny = 0: s1 == s2
// eredmeny > 0: s1 > s2

Console.WriteLine(string.Compare("alma", "barack")); // negatív (alma < barack)
Console.WriteLine(string.Compare("citrom", "alma")); // pozitív (citrom > alma)
Console.WriteLine(string.Compare("alma", "alma")); // 0

// Gyakorlati példa: belépés
string felhasznalo = "admin";
string jelszo = "Admin123";

Console.Write("Felhasználónév: ");
string inputUser = Console.ReadLine();

Console.Write("Jelszó: ");
string inputPass = Console.ReadLine();

if (inputUser.Equals(felhasznalo, StringComparison.OrdinalIgnoreCase) &&
    inputPass == jelszo)
{
    Console.WriteLine("Sikeres bejelentkezés!");
}
else
{
    Console.WriteLine("Hibás adatok!");
}
```

---

## String műveletek és metódusok

### 1. Substring - részstring kinyerése

```
string teljesNev = "Kovács János";

// Substring(startIndex) - ettől a végéig
string vezeteknev = teljesNev.Substring(0, 6); // "Kovács"
string keresztnév = teljesNev.Substring(7);    // "János"

// Substring(startIndex, length)
string resz = teljesNev.Substring(0, 3); // "Kov"

// Gyakorlati példák
string email = "pelda@email.com";
int kukacIndex = email.IndexOf('@');
string felhasznalo = email.Substring(0, kukacIndex); // "pelda"
string domain = email.Substring(kukacIndex + 1);    // "email.com"
```

```
// Utolsó 4 karakter
string kartyaszam = "1234567890123456";
string utolsoNegy = kartyaszam.Substring(kartyaszam.Length - 4); // "3456"

// Hibakezelés
string s = "Hello";
// string hiba = s.Substring(10); // ArgumentOutOfRangeException!

// Biztonságos használat
if (kukacIndex >= 0 && kukacIndex < email.Length)
{
    string user = email.Substring(0, kukacIndex);
}
```

## 2. IndexOf és LastIndexOf - keresés

```
string szoveg = "A gyors barna róka átugrik a lusta kutyán.";

// IndexOf - első előfordulás indexe
int index1 = szoveg.IndexOf('a'); // 8 (első 'a')
int index2 = szoveg.IndexOf("róka"); // 14

// Nem találja
int index3 = szoveg.IndexOf("macska"); // -1

// Keresés adott pozíciótól
int index4 = szoveg.IndexOf('a', 10); // 15 (következő 'a' a 10. indextől)

// LastIndexOf - utolsó előfordulás
int index5 = szoveg.LastIndexOf('a'); // 37 (utolsó 'a')

// Gyakorlati példa: fájl kiterjesztés
string fajlnev = "dokumentum.kepek.txt";
int pontIndex = fajlnev.LastIndexOf('.');
if (pontIndex >= 0)
{
    string kiterjesztes = fajlnev.Substring(pontIndex + 1); // "txt"
    Console.WriteLine($"Kiterjesztés: {kiterjesztes}");
}

// Ellenőrzés contains-szel
if (szoveg.IndexOf("róka") >= 0)
{
    Console.WriteLine("A szöveg tartalmazza a 'róka' szót.");
}

// Könnyebb módszer
if (szoveg.Contains("róka"))
{
}
```

```
    Console.WriteLine("A szöveg tartalmazza a 'róka' szót.");  
}
```

### 3. Contains - tartalmaz-e

```
string szoveg = "C# egy nagyszerű programozási nyelv!";  
  
// Tartalmaz-e adott szöveget  
bool tartalmaz1 = szoveg.Contains("C#");           // true  
bool tartalmaz2 = szoveg.Contains("Java");         // false  
bool tartalmaz3 = szoveg.Contains("nagyszerű");    // true  
  
// Gyakorlati példák  
string email = "teszt@gmail.com";  
if (email.Contains("@") && email.Contains("."))  
{  
    Console.WriteLine("Érvényes email formátum.");  
}  
  
// Szűrés  
string[] termek = { "Laptop", "Egér", "Billentyűzet", "Monitor" };  
string kereses = "egér";  
  
foreach (string termek in termek)  
{  
    if (termek.ToLower().Contains(kereses.ToLower()))  
    {  
        Console.WriteLine($"Találat: {termek}");  
    }  
}  
  
// Tiltott szavak ellenőrzése  
string[] tiltottSzavak = { "spam", "reklám", "vírus" };  
string komment = "Ez egy normális üzenet.";  
  
bool tartalmazTiltatot = false;  
foreach (string tiltott in tiltottSzavak)  
{  
    if (komment.ToLower().Contains(tiltott))  
    {  
        tartalmazTiltatot = true;  
        break;  
    }  
}  
  
if (tartalmazTiltatot)  
{  
    Console.WriteLine("A komment tiltott szót tartalmaz!");  
}
```

## 4. StartsWith és EndsWith

```
string fajlnev = "dokumentum.txt";

// Kezdődik-e
bool txtFajl = fajlnev.EndsWith(".txt");    // true
bool docFajl = fajlnev.EndsWith(".doc");    // false

// Végződik-e
bool dokumentum = fajlnev.StartsWith("dok"); // true
bool kep = fajlnev.StartsWith("kep");        // false

// Gyakorlati példa: URL ellenőrzés
string url = "https://www.example.com";

if (url.StartsWith("https://"))
{
    Console.WriteLine("Biztonságos kapcsolat (HTTPS)");
}
else if (url.StartsWith("http://"))
{
    Console.WriteLine("Nem biztonságos kapcsolat (HTTP)");
}

// Fájlok szűrése
string[] fajlok = { "kep1.jpg", "dokumentum.pdf", "kep2.png", "adat.txt" };

Console.WriteLine("Képfájlok:");
foreach (string fajl in fajlok)
{
    if (fajl.EndsWith(".jpg") || fajl.EndsWith(".png"))
    {
        Console.WriteLine(fajl);
    }
}

// Telefonszám ellenőrzés
string telefon = "+36301234567";
if (telefon.StartsWith("+36") || telefon.StartsWith("06"))
{
    Console.WriteLine("Magyar telefonszám");
}
```

## 5. ToUpper és ToLower - kis/nagybetű konverzió

```
string eredeti = "Hello World";

// Nagybetűssé alakítás
string nagybetuben = eredeti.ToUpper(); // "HELLO WORLD"
```

```
// Kisbetűssé alakítás
string kisbetuben = eredeti.ToLower(); // "hello world"

Console.WriteLine(eredeti); // "Hello World" (nem változott!)
Console.WriteLine(nagybetuben); // "HELLO WORLD"
Console.WriteLine(kisbetuben); // "hello world"

// Gyakorlati példák

// Összehasonlítás kis/nagybetű függetlenül
string valasz = "IGEN";
if (valasz.ToLower() == "igen")
{
    Console.WriteLine("Elfogadva");
}

// Felhasználónév normalizálás
string felhasznalo = "KoVáCs_JáNoS";
string normalizalt = felhasznalo.ToLower(); // "kovács_jános"

// Mondat nagybetűvel kezdése
string mondat = "hello világ";
if (mondat.Length > 0)
{
    string javitott = char.ToUpper(mondat[0]) + mondat.Substring(1);
    Console.WriteLine(javitott); // "Hello világ"
}

// Címsorral alakítás (minden szó nagybetűvel kezdődik)
string szoveg = "c# programozási nyelv";
string[] szavak = szoveg.Split(' ');
for (int i = 0; i < szavak.Length; i++)
{
    if (szavak[i].Length > 0)
    {
        szavak[i] = char.ToUpper(szavak[i][0]) + szavak[i].Substring(1);
    }
}
string cim = string.Join(" ", szavak); // "C# Programozási Nyelv"
```

## 6. Trim, TrimStart, TrimEnd - whitespace eltávolítás

```
string szoveg = "  Hello World  ";

// Trim - elejétől és végétől eltávolítja a szóközöket
string tiszta = szoveg.Trim(); // "Hello World"

// TrimStart - csak az elejétől
string kezdet = szoveg.TrimStart(); // "Hello World  "

// TrimEnd - csak a végétől
```



```
string vege = szoveg.TrimEnd(); // "    Hello World"

Console.WriteLine($"'{szoveg}'"); // '    Hello World    '
Console.WriteLine($"'{tiszta}'"); // 'Hello World'

// Gyakorlati példák

// Felhasználói input tisztítása
Console.Write("Add meg a neved: ");
string nev = Console.ReadLine().Trim(); // Eltávolítja a felesleges szóközöket

// Többféle whitespace karakter eltávolítása
string bemenet = "\t\n Szöveg \n\t";
string tisztitott = bemenet.Trim(); // "Szöveg"

// Specifikus karakterek eltávolítása
string adat = "###Hello###";
string eredmeny = adat.Trim('#'); // "Hello"

string szamok = "0001234000";
string szam = szamok.Trim('0'); // "1234"

// Email validálás előkészítése
string email = "  teszt@example.com  ";
email = email.Trim().ToLower();
if (email.Contains("@"))
{
    Console.WriteLine("Email formátum rendben.");
}
```

## 7. Replace - csere

```
string eredeti = "Hello World";

// Karakter cseréje
string csere1 = eredeti.Replace('o', 'a'); // "Hella World"

// String cseréje
string csere2 = eredeti.Replace("World", "Universe"); // "Hello Universe"
string csere3 = eredeti.Replace("xyz", "abc"); // "Hello World" (nincs változás)

// Eltávolítás (üres stringre cseréljük)
string szoveg = "A-B-C-D";
string eredmeny = szoveg.Replace("-", ""); // "ABCD"

// Gyakorlati példák

// Szóköz eltávolítása
string telefonszam = "06 30 123 4567";
string tiszta = telefonszam.Replace(" ", ""); // "06301234567"
```

```
// Cenzúrázás
string komment = "Ez egy rossz szó tartalma.";
string cenzurazott = komment.Replace("rossz", "*****");

// Több csere egymás után
string szoveg2 = "alma, körte, barack";
szoveg2 = szoveg2.Replace("alma", "citrom");
szoveg2 = szoveg2.Replace(", ", ";");
Console.WriteLine(szoveg2); // "citrom; körte; barack"

// Fájlnev normalizálás
string fajlnev = "Dokumentum 2024.txt";
fajlnev = fajlnev.Replace(" ", "_"); // "Dokumentum_2024.txt"

// Többszörös szóköz helyettesítése eggyel
string tobbszoros = "Hello    World";
while (tobbszoros.Contains(" "))
{
    tobbszoros = tobbszoros.Replace(" ", " ");
}
Console.WriteLine(tobbszoros); // "Hello World"
```

## 8. Split - szétválasztás

```
string mondat = "alma,körte,barack,szilva";

// Szétválasztás egy karakter alapján
string[] gyumolcsok = mondat.Split(',');
// gyumolcsok = ["alma", "körte", "barack", "szilva"]

foreach (string gyumolcs in gyumolcsok)
{
    Console.WriteLine(gyumolcs);
}

// Szétválasztás több karakter alapján
string szoveg = "alma;körte,barack-szilva";
string[] elemek = szoveg.Split(',', ';', '-');
// elemek = ["alma", "körte", "barack", "szilva"]

// Szétválasztás szóköz alapján
string mondat2 = "Ez egy próba mondat";
string[] szavak = mondat2.Split(' ');

// Üres elemek kihagyása
string adat = "egy,,kettő,,három";
string[] reszek = adat.Split(new char[] { ',', ' ' },
StringSplitOptions.RemoveEmptyEntries);
// reszek = ["egy", "kettő", "három"]
```

```
// Gyakorlati példák

// CSV feldolgozás
string csv = "Kovács János,30,Budapest";
string[] adatok = csv.Split(',');
string nev = adatok[0]; // "Kovács János"
int életkor = int.Parse(adatok[1]); // 30
string varos = adatok[2]; // "Budapest"

Console.WriteLine($"Név: {nev}, Életkor: {életkor}, Város: {varos}");

// URL paraméterek feldolgozása
string url = "https://example.com?name=John&age=30&city=NY";
int kezdet = url.IndexOf('?');
if (kezdet >= 0)
{
    string parameterok = url.Substring(kezdet + 1);
    string[] paramParak = parameterok.Split('&');

    foreach (string param in paramParak)
    {
        string[] kulcsErtek = param.Split('=');
        Console.WriteLine($"{kulcsErtek[0]}: {kulcsErtek[1]}");
    }
}

// Mondat szavakra bontása és számolás
string hosszuMondat = "A gyors barna róka átugrik a lusta kutyán";
string[] mondat3Szavak = hosszuMondat.Split(' ');
Console.WriteLine($"A mondatban {mondat3Szavak.Length} szó van.");

// Sorok feldolgozása
string tobbsoros = "első sor\nmásodik sor\nharmadik sor";
string[] sorok = tobbsoros.Split('\n');
int sorSzam = 1;
foreach (string sor in sorok)
{
    Console.WriteLine($"{sorSzam}. {sor}");
    sorSzam++;
}
```

## 9. Join - összefűzés

```
string[] szavak = { "C#", "Java", "Python", "JavaScript" };

// Összefűzés elválasztó karakterrel
string nyelvek = string.Join(", ", szavak);
Console.WriteLine(nyelvek); // "C#, Java, Python, JavaScript"

// Különböző elválasztók
string vesszos = string.Join(",", szavak); // "C#,Java,Python,JavaScript"
```

```

string ujsoros = string.Join("\n", szavak); // Minden elem új sorban
string szokozos = string.Join(" ", szavak); // "C# Java Python JavaScript"

// Számok összefűzése
int[] szamok = { 1, 2, 3, 4, 5 };
string szamLista = string.Join("-", szamok);
Console.WriteLine(szamLista); // "1-2-3-4-5"

// Gyakorlati példák

// CSV generálás
string nev = "Kovács János";
int életkor = 30;
string varos = "Budapest";
string csvSor = string.Join(",", nev, életkor, varos);
Console.WriteLine(csvSor); // "Kovács János,30,Budapest"

// URL építés
string[] utvonalReszek = { "users", "123", "profile" };
string url = "https://api.example.com/" + string.Join("/", utvonalReszek);
Console.WriteLine(url); // "https://api.example.com/users/123/profile"

// Lista formázás
string[] teendok = { "Bevásárlás", "Takarítás", "Tanulás" };
string listaStr = string.Join("\n- ", teendok);
Console.WriteLine("Teendők:\n- " + listaStr);
/* Kimenet:
Teendők:
- Bevásárlás
- Takarítás
- Tanulás
*/

// Szűrt elemek összefűzése
string[] nevek = { "Anna", "Béla", "", "Cecil", null, "Dóra" };
string[] szurtNevak = nevek.Where(n => !string.IsNullOrEmpty(n)).ToArray();
string osszefuzott = string.Join(", ", szurtNevak);
Console.WriteLine(osszefuzott); // "Anna, Béla, Cecil, Dóra"

```

## 10. PadLeft és PadRight - kitöltés

```

string szam = "42";

// PadLeft - balról kitölt
string bal = szam.PadLeft(5, '0'); // "00042"
string bal2 = szam.PadLeft(5); // " 42" (szóközzel)

// PadRight - jobbról kitölt
string jobb = szam.PadRight(5, '0'); // "42000"
string jobb2 = szam.PadRight(5); // "42 " (szóközzel)

```

```
// Gyakorlati példák

// Számok formázása
for (int i = 1; i <= 10; i++)
{
    string formatum = i.ToString().PadLeft(2, '0');
    Console.WriteLine($"Elem {formatum}");
}
/* Kimenet:
Elem 01
Elem 02
...
Elem 10
*/

// Táblázat készítés
string[] nevek = { "Anna", "Béla", "Cecil" };
int[] pontok = { 95, 87, 92 };

Console.WriteLine("Név".PadRight(10) + "Pontszám");
Console.WriteLine("".PadRight(20, '-'));

for (int i = 0; i < nevek.Length; i++)
{
    string sor = nevek[i].PadRight(10) + pontok[i];
    Console.WriteLine(sor);
}
/* Kimenet:
Név      Pontszám
-----
Anna      95
Béla      87
Cecil     92
*/

// Bankszámlaszám formázás
string szamlaszam = "1234567890123456";
string formatum2 = "";
for (int i = 0; i < szamlaszam.Length; i += 4)
{
    formatum2 += szamlaszam.Substring(i, 4) + " ";
}
Console.WriteLine(formatum2.Trim()); // "1234 5678 9012 3456"

// Igazítás
string balra = "Balra".PadRight(20, '.'); // "Balra....."
string jobbra = "Jobbra".PadLeft(20, '.'); // ".....Jobbra"
string kozep = "Középre";
int osszHossz = 20;
int padding = (osszHossz - kozep.Length) / 2;
string kozepre = kozep.PadLeft(kozep.Length + padding).PadRight(osszHossz);
Console.WriteLine($"|{balra}|");
Console.WriteLine($"|{jobbra}|");
Console.WriteLine($"|{kozepre}|");
```

## String interpoláció és formázás

### String interpoláció (C# 6.0+)

Az interpolált stringek `$` jellel kezdődnek és `{}` között kifejezéseket tartalmaznak.

```
string nev = "János";
int életkor = 25;

// Régi módszer - összefűzés
string uzenet1 = "Szia, " + nev + "! " + életkor + " éves vagy.";

// String.Format
string uzenet2 = string.Format("Szia, {0}! {1} éves vagy.", nev, életkor);

// String interpoláció (modern, ajánlott)
string uzenet3 = $"Szia, {nev}! {életkor} éves vagy.";

Console.WriteLine(uzenet3); // "Szia, János! 25 éves vagy."

// Kifejezések interpolációban
int a = 10;
int b = 20;
Console.WriteLine($"{a} + {b} = {a + b}"); // "10 + 20 = 30"

// Metódus hívás
string szoveg = "hello";
Console.WriteLine($"Nagybetűvel: {szoveg.ToUpper()}"); // "Nagybetűvel: HELLO"

// Ternáris operátor
int pontszam = 85;
Console.WriteLine($"Eredmény: {(pontszam >= 60 ? "Megfelelt" : "Nem felelt meg")}");

// Objektum tulajdonság
DateTime most = DateTime.Now;
Console.WriteLine($"Mai dátum: {most.Year}-{most.Month}-{most.Day}");
```

### Számok formázása

```
double szam = 1234.5678;

// Általános formázás
Console.WriteLine($"{szam}"); // 1234.5678

// Tizedesjegyek száma
Console.WriteLine($"{szam:F2}"); // 1234.57 (2 tizedesjegy)
```

```

Console.WriteLine($"{szam:F0}");           // 1235 (kerekítés, 0 tizedesjegy)

// Pénznem formátum
decimal ar = 15000.5m;
Console.WriteLine($"{ar:C}");              // 15 000,50 Ft (rendszer nyelvtől függ)
Console.WriteLine($"{ar:C0}");             // 15 001 Ft (kerekítve)

// Ezres elválasztó
int nagy = 1000000;
Console.WriteLine($"{nagy:N}");            // 1 000 000,00
Console.WriteLine($"{nagy:N0}");           // 1 000 000

// Százalék
double arany = 0.856;
Console.WriteLine($"{arany:P}");           // 85,60%
Console.WriteLine($"{arany:P0}");          // 86%

// Nullákkal kitöltés
int szam2 = 42;
Console.WriteLine($"{szam2:D5}");          // 00042
Console.WriteLine($"{szam2:00000}");       // 00042

// Tudományos jelölés
double kicsi = 0.000123;
Console.WriteLine($"{kicsi:E}");           // 1,230000E-004

// Hexadecimális
int szam3 = 255;
Console.WriteLine($"{szam3:X}");           // FF
Console.WriteLine($"{szam3:X4}");          // 00FF

// Gyakorlati példák
decimal nettoAr = 10000m;
decimal afa = nettoAr * 0.27m;
decimal bruttoAr = nettoAr + afa;

Console.WriteLine("=== SZÁMLA ===");
Console.WriteLine($"Nettó ár: {nettoAr,10:N0} Ft");
Console.WriteLine($"ÁFA (27%): {afa,10:N0} Ft");
Console.WriteLine($"",10}-----");
Console.WriteLine($"Bruttó ár: {bruttoAr,10:N0} Ft");

```

## Dátum és idő formázása

```

DateTime most = DateTime.Now;

// Alapértelmezett
Console.WriteLine($"{most}");

// Teljes dátum és idő
Console.WriteLine($"{most:F}");           // 2024. január 15. 14:30:45

```

```
// Hosszú dátum
Console.WriteLine($"{most:D}");           // 2024. január 15.

// Rövid dátum
Console.WriteLine($"{most:d}");           // 2024. 01. 15.

// Hosszú idő
Console.WriteLine($"{most:T}");           // 14:30:45

// Rövid idő
Console.WriteLine($"{most:t}");           // 14:30

// Egyéni formátum
Console.WriteLine($"{most:yyyy.MM.dd}");           // 2024.01.15
Console.WriteLine($"{most:yyyy-MM-dd HH:mm:ss}"); // 2024-01-15 14:30:45
Console.WriteLine($"{most:dd/MM/yyyy}");           // 15/01/2024
Console.WriteLine($"{most:MMMM dd, yyyy}");        // január 15, 2024
Console.WriteLine($"{most:dddd}");                 // hétfő

// Gyakorlati példa
string nev = "Kovács János";
DateTime szuletes = new DateTime(1995, 5, 20);
int kor = DateTime.Now.Year - szuletes.Year;

Console.WriteLine($"Név: {nev}");
Console.WriteLine($"Születési dátum: {szuletes:yyyy. MMMM dd.}");
Console.WriteLine($"Életkor: {kor} év");
Console.WriteLine($"Mai dátum: {DateTime.Now:yyyy.MM.dd HH:mm}");
```

## Igazítás és szélesség

```
// Szélesség megadása
string nev = "Anna";
int szam = 42;

// Jobbra igazítás (pozitív szám)
Console.WriteLine($"|{nev,10}|"); // |          Anna|
Console.WriteLine($"|{szam,10}|"); // |          42|

// Balra igazítás (negatív szám)
Console.WriteLine($"|{nev,-10}|"); // |Anna          |
Console.WriteLine($"|{szam,-10}|"); // |42           |

// Kombinálás formázással
decimal ar = 1234.56m;
Console.WriteLine($"|{ar,15:N2}|"); // "          1 234,56" (jobbra igazítva)

// Táblázat készítése
string[] nevek = { "Anna", "Béla", "Cecil" };
int[] pontok = { 95, 87, 100 };
```



```
double[] atlagok = { 4.8, 4.1, 5.0 };

Console.WriteLine($"{ "Név",-10} {"Pont",5} {"Átlag",6}");
Console.WriteLine(new string('-', 25));

for (int i = 0; i < nevek.Length; i++)
{
    Console.WriteLine($"{nevek[i],-10} {pontok[i],5} {atlagok[i],6:F1}");
}
/* Kimenet:
Név          Pont  Átlag
-----
Anna          95    4,8
Béla          87    4,1
Cecil        100    5,0
*/
```

## StringBuilder - hatékony string építés

A **StringBuilder** **megváltoztatható** (mutable) string-kezelést tesz lehetővé, sokkal hatékonyabb, ha sok módosítást végzünk.

### Miért StringBuilder?

```
// ROSSZ - sok string példányosítás (lassú)
string eredmeny = "";
for (int i = 0; i < 10000; i++)
{
    eredmeny += i.ToString(); // Minden iterációban új string jön létre!
}

// JÓ - StringBuilder használata (gyors)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++)
{
    sb.Append(i); // Hatékonyan módosítja ugyanazt az objektumot
}
string eredmeny2 = sb.ToString();

// A string immutable (megváltozhatatlan):
string s1 = "Hello";
s1 += " World"; // Új string objektum jön létre, s1 az újra mutat
// Az eredeti "Hello" string memóriában marad szemétként

// A StringBuilder mutable (megváltoztatható):
StringBuilder sb2 = new StringBuilder("Hello");
sb2.Append(" World"); // Ugyanazt az objektumot módosítja, nem jön létre új
```

### StringBuilder alapok

```
using System.Text; // Fontos!

// Létrehozás
StringBuilder sb1 = new StringBuilder();
StringBuilder sb2 = new StringBuilder("Kezdő szöveg");
StringBuilder sb3 = new StringBuilder(100); // Kezdő kapacitással

// Append - hozzáfűzés a végére
StringBuilder sb = new StringBuilder();
sb.Append("Hello");
sb.Append(" ");
sb.Append("World");
string eredmeny = sb.ToString(); // "Hello World"

// Láncolás (fluent interface)
StringBuilder sb4 = new StringBuilder();
sb4.Append("Egy").Append(" ").Append("mondat").Append(".");
Console.WriteLine(sb4.ToString()); // "Egy mondat."

// AppendLine - új sorral hozzáfűz
StringBuilder sb5 = new StringBuilder();
sb5.AppendLine("Első sor");
sb5.AppendLine("Második sor");
sb5.AppendLine("Harmadik sor");
Console.WriteLine(sb5.ToString());

// AppendFormat - formázott hozzáfűzés
StringBuilder sb6 = new StringBuilder();
string nev = "János";
int kor = 25;
sb6.AppendFormat("Név: {0}, Kor: {1}", nev, kor);

// String interpolációval (C# 10+)
StringBuilder sb7 = new StringBuilder();
sb7.Append($"Név: {nev}, Kor: {kor}");
```

## StringBuilder műveletek

```
StringBuilder sb = new StringBuilder("Hello World");

// Insert - beszúrás adott pozícióra
sb.Insert(5, ","); // "Hello, World"

// Remove - törlés
sb.Remove(5, 1); // "HelloWorld" (törli a vesszőt)

// Replace - csere
sb.Replace("World", "Universe"); // "HelloUniverse"
sb.Replace('o', 'a'); // "HellUniverse" -> "HellUniverse"
```

```
// Clear - teljes törlés
sb.Clear();           // ""

// Length - hossz
sb.Append("Test");
Console.WriteLine(sb.Length); // 4

// Indexelés
sb[0] = 'B';          // "Best"

// ToString - string-é alakítás
string vegeredmeny = sb.ToString();
```

## Gyakorlati StringBuilder példák

```
// HTML generálás
StringBuilder html = new StringBuilder();
html.AppendLine("<html>");
html.AppendLine("<head><title>Oldal címe</title></head>");
html.AppendLine("<body>");
html.AppendLine("  <h1>Üdvözljük!</h1>");
html.AppendLine("  <p>Ez egy generált HTML oldal.</p>");
html.AppendLine("</body>");
html.AppendLine("</html>");

Console.WriteLine(html.ToString());

// CSV fájl generálás
StringBuilder csv = new StringBuilder();
csv.AppendLine("Név,Életkor,Város");

string[] nevek = { "Anna", "Béla", "Cecil" };
int[] korok = { 25, 30, 35 };
string[] varosok = { "Budapest", "Debrecen", "Szeged" };

for (int i = 0; i < nevek.Length; i++)
{
    csv.AppendLine($"{nevek[i]},{korok[i]},{varosok[i]}");
}

Console.WriteLine(csv.ToString());

// SQL lekérdezés építés
StringBuilder sql = new StringBuilder();
sql.Append("SELECT * FROM Users ");
sql.Append("WHERE Active = 1 ");
sql.Append("AND Age >= 18 ");
sql.Append("ORDER BY Name");

string lekerdez = sql.ToString();
```

```
// Nagy szöveg generálás
StringBuilder nagy = new StringBuilder();
for (int i = 1; i <= 100; i++)
{
    nagy.AppendLine($"Ez a {i}. sor.");
}

// JSON-szerű struktúra
StringBuilder json = new StringBuilder();
json.AppendLine("{");
json.AppendLine($"  \"name\": \"János\",");
json.AppendLine($"  \"age\": 30,");
json.AppendLine($"  \"city\": \"Budapest\"");
json.AppendLine("}");

// Szöveg fordítás (reverse)
string eredeti = "Hello";
StringBuilder fordított = new StringBuilder();
for (int i = eredeti.Length - 1; i >= 0; i--)
{
    fordított.Append(eredeti[i]);
}
Console.WriteLine(fordított.ToString()); // "olleH"
```

---

## String Helper metódusok

### IsNullOrEmpty és IsNullOrWhiteSpace

```
string null1 = null;
string ures = "";
string szokozes = " ";
string valami = "szöveg";

// IsNullOrEmpty - null vagy üres
Console.WriteLine(string.IsNullOrEmpty(null1)); // true
Console.WriteLine(string.IsNullOrEmpty(ures)); // true
Console.WriteLine(string.IsNullOrEmpty(szokozes)); // false
Console.WriteLine(string.IsNullOrEmpty(valami)); // false

// IsNullOrWhiteSpace - null, üres vagy csak szóköz
Console.WriteLine(string.IsNullOrWhiteSpace(null1)); // true
Console.WriteLine(string.IsNullOrWhiteSpace(ures)); // true
Console.WriteLine(string.IsNullOrWhiteSpace(szokozes)); // true
Console.WriteLine(string.IsNullOrWhiteSpace(valami)); // false

// Gyakorlati használat
string nev = Console.ReadLine();

if (string.IsNullOrWhiteSpace(nev))
{
```

```
        Console.WriteLine("A név megadása kötelező!");
    }
    else
    {
        Console.WriteLine($"Üdv, {nev}!");
    }
}
```

## Egyéb hasznos metódusok

```
// Concat - összefűzés
string s1 = string.Concat("Hello", " ", "World"); // "Hello World"

// Copy - másolat
string eredeti = "Hello";
string masol = string.Copy(eredeti);

// Empty - üres string konstans
string ures = string.Empty; // Ajánlott "" helyett

// Format - formázás
string formaz = string.Format("Név: {0}, Kor: {1}", "János", 25);

// IsInterned - string pool ellenőrzés
string lit = "Hello";
string interned = string.IsInterned(lit);
```

---

## Összefoglaló gyakorlat

```
using System;
using System.Text;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== STRING MŰVELETEK ÖSSZEFOGLALÓ ===\n");

        // Felhasználói adatok bekérése
        Console.Write("Add meg a neved: ");
        string nev = Console.ReadLine().Trim();

        Console.Write("Add meg az email címed: ");
        string email = Console.ReadLine().Trim().ToLower();

        Console.Write("Add meg a telefonszámod: ");
        string telefon = Console.ReadLine().Trim();

        // Validálás
```

```
bool ervenyesNev = !string.IsNullOrEmpty(nev);
bool ervenyesEmail = email.Contains("@") && email.Contains(".");
bool ervenyesTelefon = telefon.Length >= 9;

if (ervenyesNev && ervenyesEmail && ervenyesTelefon)
{
    // Adatok formázása
    telefon = telefon.Replace(" ", "").Replace("-", "");

    // Üdvözlő üzenet készítése StringBuilder-rel
    StringBuilder uzenet = new StringBuilder();
    uzenet.AppendLine("=== REGISZTRÁCIÓ SIKERES ===");
    uzenet.AppendLine($"Név: {nev}");
    uzenet.AppendLine($"Email: {email}");
    uzenet.AppendLine($"Telefon: {telefon}");
    uzenet.AppendLine();
    uzenet.AppendLine($"Regisztráció dátuma: {DateTime.Now:yyyy.MM.dd HH:mm}");

    Console.WriteLine("\n" + uzenet.ToString());
}
else
{
    Console.WriteLine("\nHibás adatok!");
    if (!ervenyesNev) Console.WriteLine("- Név megadása kötelező");
    if (!ervenyesEmail) Console.WriteLine("- Érvénytelen email cím");
    if (!ervenyesTelefon) Console.WriteLine("- Érvénytelen telefonszám");
}
}
```

---

## Összefoglalás

### String tulajdonságok

- **Length:** String hossza

### Keresés

- **IndexOf/LastIndexOf:** Karakter/string keresése
- **Contains:** Tartalmaz-e
- **StartsWith/EndsWith:** Kezdődik/Végződik

### Módosítás (új stringet ad vissza!)

- **Substring:** Részstring kivágása
- **ToUpper/ToLower:** Kis/nagybetűvé alakítás
- **Trim/TrimStart/TrimEnd:** Whitespace eltávolítás
- **Replace:** Csere
- **PadLeft/PadRight:** Kitöltés

## Szétválasztás/Összefűzés

- **Split:** Szétválasztás
- **Join:** Összefűzés

## String formázás

- **String interpoláció:** `$"Szöveg {változó}"`
- **Formázó kódok:** `:F2`, `:C`, `:N`, `:P`, stb.

## StringBuilder

- **Mutable** string kezelés
- Hatékony sok módosításnál
- **Append/AppendLine:** Hozzáfűzés
- **Insert/Remove/Replace:** Módosítás
- **ToString():** String-é alakítás

### Fontos:

- A string **immutable** - minden "módosítás" új stringet hoz létre
- StringBuilder használata sok módosításnál
- `IsNullOrEmpty` / `IsNullOrWhiteSpace` használata validáláshoz
- String interpoláció modern és olvasható

---

*C# Alapok Tananyag - 5. Fejezet*